

La STL de C++

Los contratos que la biblioteca ya trae hechos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Usar `vector`, `string`, `stack` y `queue` de la biblioteca estándar como implementaciones de los contratos vistos: crear, consultar, modificar y recorrer.
- Traducir una función escrita contra el TAD `Lista` a una escrita contra `vector`, y explicar qué cambió y qué no.
- Distinguir el acceso vigilado (`at`) del no vigilado (`[]`), y el paso por valor del paso por referencia en un contenedor.
- Aplicar `sort`, `find`, `binary_search`, `reverse`, `count`, `min_element` y `max_element` sobre un vector, con un criterio de orden propio cuando haga falta.
- Decir qué cuesta en tiempo y en espacio cada operación de esos contenedores y cada algoritmo, y escoger la operación barata cuando el contrato ofrece dos caminos.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: implementar soluciones a problemas pequeños con las estructuras que provee el lenguaje.
- OA2: escoger el contenedor por el costo de sus operaciones.
- OA3: leer documentación técnica en inglés y traducirla a un contrato.

Referencia bibliográfica

B. Stroustrup, *The C++ Programming Language*, 4.^a ed., 2013, capítulo 31 (contenedores de la STL) y capítulo 32 (algoritmos de la STL). Documentación en línea:
<https://en.cppreference.com/w/cpp/container/vector>.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Tres contratos y un área del programador que no los mira por dentro I

Lo que quedó

- Lista, Pila y Cola son secuencias con distinta disciplina de acceso: por posición, por el tope, por el frente.
- mezclar, revertir, balanceado, particionar, la papa caliente: todas escritas solo con el contrato, sobre una implementación provisional que nadie vio.

Pregunta

Si la implementación no la vio nadie, ¿quién la escribió? ¿Y quién la escribe cuando el programa es de verdad?

- Hoy la respuesta es: la biblioteca estándar de C++. Trae esos tres contratos, y varios más, ya implementados, probados y medidos.

Tres contratos y un área del programador que no los mira por dentro II

- Lo que aprendimos a escribir `contra lista.h` se escribe igual contra ella. Esa es la prueba de que valió la pena separar contrato de representación.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Tres piezas I

Definición (STL, Standard Template Library)

*La parte de la biblioteca estándar de C++ que provee **contenedores** (estructuras que guardan elementos), **algoritmos** (operaciones sobre secuencias, como ordenar o buscar) e **iteradores** (la forma de señalar posiciones dentro de un contenedor, común a todos ellos). Stroustrup, capítulo 31.*

Contenedores que veremos en el curso

- Hoy: vector, string, stack, queue.
- Más adelante: list, deque, map, set, priority_queue, cada uno cuando hayamos construido a mano la estructura que lo respalda.

Tres piezas II

- Los iteradores tienen sesión propia. Hoy solo usamos dos: `v.begin()` y `v.end()`, que señalan el principio y el final del contenedor.

Todo vive en el espacio de nombres std

```
#include <string>

string s = "hola";    // no compila
```

Lo que dice el compilador

error: 'string' was not declared in this scope
note: suggested alternatives: 'std::string'

- El #include trajo la declaración, pero cada nombre de la biblioteca estándar está declarado dentro de un **espacio de nombres** llamado std, y desde afuera solo se ve con su nombre completo: std::string. Sin el #include el nombre no existe; sin el std, existe pero no se alcanza. Stroustrup, capítulo 14.

Todo vive en el espacio de nombres std II

```
std::string s = "hola";    // el nombre completo, cada vez  
  
using std::string;         // se trae ese nombre y nada mas  
string t = "hola";  
  
using namespace std;      // se traen todos, de una vez
```

En el curso se escribe `using std::nombre`

Una línea por cada nombre de la biblioteca que el programa usa, debajo de los `#include`. El encabezado dice qué se toma prestado, y nada más entra.

Todo vive en el espacio de nombres std III

- Con `using namespace std` entran cientos de nombres. Si el programa declara una variable `count`, o `size`, cada uso choca con la función de la biblioteca que se llama igual: `error: reference to 'count' is ambiguous`. Con `using std::vector` no entra nada más.
- Lo que viene de C, como `fgets` de `<cstdio>`, se trae igual: `using std::fgets`.

Un contrato, cualquier tipo de elemento I

```
#include <vector>
#include <string>
using std::vector;
using std::string;

vector<int> enteros;
vector<double> notas;
vector<string> nombres;
vector<Estudiante> grupo;
```

Plantilla

vector no es un tipo: es un molde de tipos. Al escribir `vector<int>` el compilador produce la clase para enteros; con `vector<Estudiante>`, la clase para estudiantes. El contrato es el mismo y el código de la biblioteca es uno solo.

Un contrato, cualquier tipo de elemento II

- En `lista.h` el tipo del elemento era un `typedef` fijo: una lista de `int` y nada más. El `<int>` entre ángulos resuelve eso.
- Lo que se gana: no escribir la implementación. Lo que se cede: no verla. El costo de cada operación hay que leerlo en la documentación, y lo vamos a verificar cuando construyamos las nuestras.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector**
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Crear, medir, leer I

```
vector<int> v;           // vacío
vector<int> w(5, 0);     // cinco ceros
vector<int> a = {4, 5, 9}; // con valores

int n = a.size();       // 3
int x = a[1];           // 5, sin vigilar
int y = a.at(1);        // 5, vigilado
bool e = v.empty();     // true
```

Dos formas de leer la posición p

$v[p]$ y $v.at(p)$ responden lo mismo cuando $0 \leq p < \text{size}()$.
Difieren en qué pasa cuando no.

- `size()` devuelve un entero sin signo. Al compararlo con un `int` el compilador avisa; en el curso se escribe `(int)` `v.size()` y listo.

La precondition, vigilada por la biblioteca l

```
int main() {  
    vector<int> v;  
    v.push_back(7);  
    v.push_back(2);  
    printf("v.at(1) = %d\n", v.at(1));  
    printf("v.at(5) = %d\n", v.at(5));  
    printf("esta linea no se alcanza\n");  
    return 0;  
}
```

Salida real

```
v.at(1) = 2  
terminate called after throwing an instance of  
↳ 'std::out_of_range'  
    what():  vector::_M_range_check: __n (which is 5)  
            >= this->size() (which is 2)
```

La precondition, vigilada por la biblioteca II

- `at` hace lo que hacía el `assert` de `lista.h`: comprueba $p < \text{size}()$ y detiene el programa donde nace el error, diciendo qué índice y qué tamaño.
- `v[5]` sobre ese mismo vector **no** avisa: lee memoria que no es del vector. A veces sale basura, a veces el programa sigue como si nada, a veces se cae más adelante. Es la misma situación del `struct` expuesto con `den = 0`: ninguna regla del lenguaje se violó y el resultado no significa nada.
- `[]` es más rápido porque no comprueba. Se usa cuando el programa ya aseguró el rango, como en un ciclo de 0 a $\text{size}() - 1$.

Las operaciones, una por una I

```
int main() {  
    vector<int> v;  
    imprimir(v);  
    v.push_back(5);  
    v.push_back(8);  
    imprimir(v);  
    v.insert(v.begin() + 1, 7);  
    imprimir(v);  
    v.erase(v.begin() + 0);  
    imprimir(v);  
    v[1] = 2;  
    imprimir(v);  
    printf("size: %d, empty: %d\n", (int) v.size(), (int)  
        ↪ v.empty());  
    return 0;  
}
```

Las operaciones, una por una II

Salida real, y el contrato de Lista al lado

con vector	con el TAD Lista	estado
<code>push_back(5), push_back(8)</code>	<code>agregar(5), agregar(8)</code>	<code>< 5 8 ></code>
<code>insert(v.begin() + 1, 7)</code>	<code>insertar(1, 7)</code>	<code>< 5 7 8 ></code>
<code>erase(v.begin() + 0)</code>	<code>eliminar(0)</code>	<code>< 7 8 ></code>
<code>v[1] = 2</code>	<code>asignar(1, 2)</code>	<code>< 7 2 ></code>
<code>size(), empty()</code>	<code>tamano(), vacia()</code>	<code>2, 0</code>

- La misma traza de la sesión de contratos, con otros nombres. `v.begin() + p` es la manera de decir “la posición p ” a `insert` y a `erase`.
- `insert` admite $p = \text{size}()$ y `erase` no, exactamente como `insertar` y `eliminar`. Ninguna de las dos vigila: fuera de rango es como `[]`.

Por valor o por referencia I

```
void duplicarCopia(vector<int> v) {  
    int p = 0;  
    while (p < (int) v.size()) {  
        v[p] = 2 * v[p];  
        p = p + 1;  
    }  
}
```

```
void duplicar(vector<int> &v) {  
    int p = 0;  
    while (p < (int) v.size()) {  
        v[p] = 2 * v[p];  
        p = p + 1;  
    }  
}
```

Por valor o por referencia II

Pregunta

Con `v = {3, 4}`, ¿qué imprime `main` después de llamar a cada una?

Salida real

```
tras duplicarCopia: 3 4  
tras duplicar:      6 8
```

- Sin `&`, la función recibe una **copia completa** del vector: sus cambios se pierden al volver, y copiar costó tantos pasos como elementos había.
- Con `&`, trabaja sobre el mismo vector. Es la regla que ya usábamos con `Lista &l`: los contenedores se pasan por referencia salvo que se quiera la copia a propósito.

El área del programador, ahora sobre vector l

```
// copia al final de r los elementos de l desde la posición p  
void copiarDesde(vector<int> &r, vector<int> &l, int p) {  
    while (p < (int) l.size()) {  
        r.push_back(l[p]);  
        p = p + 1;  
    }  
}
```

El área del programador, ahora sobre vector II

```
// devuelve un vector nuevo con los mismos elementos en orden  
↪ inverso  
vector<int> revertir(vector<int> &l) {  
    vector<int> r;  
    int p = 0;  
    while (p < (int) l.size()) {  
        r.insert(r.begin(), l[p]);  
        p = p + 1;  
    }  
    return r;  
}
```

El área del programador, ahora sobre vector III

```
vector<int> mezclar(vector<int> &a, vector<int> &b) {  
    vector<int> r;  
    int i = 0;  
    int j = 0;  
    while (i < (int) a.size() && j < (int) b.size()) {  
        if (a[i] <= b[j]) {  
            r.push_back(a[i]);  
            i = i + 1;  
        } else {  
            r.push_back(b[j]);  
            j = j + 1;  
        }  
    }  
    copiarDesde(r, a, i);  
    copiarDesde(r, b, j);  
    return r;  
}
```

El área del programador, ahora sobre vector IV

Qué cambió y qué no

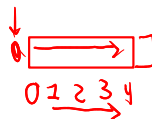
antes	ahora
Lista &a	vector<int> &a
a.tamano()	(int) a.size()
a.obtener(i)	a[i]
r.agregar(x)	r.push_back(x)
r.insertar(0, x)	r.insert(r.begin(), x)

- Cambiaron nombres. No cambió una sola decisión: las dos posiciones, el $\leq$ del empate, el resto copiado con la función auxiliar. Salida real: $\langle 1 \ 4 \ 5 \ 5 \ 8 \ 9 \rangle$ y $\langle 9 \ 8 \ 5 \ 5 \ 4 \ 1 \rangle$, las mismas de la sesión de contratos.
- Quien escribió contra el contrato ya sabía programar con vector sin haberlo visto.

El área del programador, ahora sobre vector V

- Costos, con n el total de elementos: mezclar $\Theta(n)$ en tiempo (n push_back de $O(1)$) y $\Theta(n)$ de espacio, el vector nuevo. revertir con insert al frente, $\Theta(n^2)$: la k -ésima inserción corre $k - 1$ elementos. Con push_back desde el final sería $\Theta(n)$ con el mismo espacio.

1) Insertar uno $\rightarrow O(n)$
2) n elementos $\rightarrow$



Un vector de vectores I



```
vector<vector<int>> m(3, vector<int>(4, 0));  
m[1][2] = 6;           // fila 1, columna 2  
m[1].push_back(99);    // la fila 1 ahora tiene 5 columnas  
int filas = m.size();  
int columnasFila1 = m[1].size();
```

- m es un vector cuyos elementos son vectores: una matriz de 3 filas y 4 columnas, todas en cero.
- A diferencia del arreglo bidimensional de C, cada fila puede cambiar de tamaño por su cuenta. Es cómodo y a veces es una trampa: un recorrido que asume filas iguales se cae en la que creció. *m[i].size() columnas*
- Salida real del programa que la llena con $i \cdot 4 + j$: la fila 1 imprime 4 5 6 7 99.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string**
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Una cadena es un contenedor de caracteres I

```
#include <string>
using std::string;

string s = "estructura";
string t = s + " de datos"; // 19 caracteres
char c = s[0];              // 'e'
int n = s.size();           // 10
bool igual = (s == "pila"); // false
bool antes = (s < "pila");  // true: orden alfabetico
printf("%s\n", t.c_str());  // para printf, c_str()
```

- Tiene size, [], at, push_back: el mismo contrato de vector<char>, más lo propio de texto: concatenar con +, comparar con == y <.
- printf no sabe de string: se le pasa s.c_str(), el arreglo de char que la cadena guarda por dentro.

Una cadena es un contenedor de caracteres II

```
int vocales(string s) {  
    int c = 0;  
    int i = 0;  
    while (i < (int) s.size()) {  
        if (s[i] == 'a' || s[i] == 'e' || s[i] == 'i'  
            || s[i] == 'o' || s[i] == 'u') {  
            c = c + 1;  
        }  
        i = i + 1;  
    }  
    return c;  
}
```

- El recorrido es el de todo vector: posición de 0 a `size()` - 1.
Salida real: vocales de estructura: 4.

Ejercicio en parejas: al revés I

Enunciado

Escriba `alReves(s)`, que devuelve una cadena nueva con los caracteres de `s` en orden inverso, y úsela para decidir si reconocer se lee igual al derecho y al revés.

Pistas

- Es revertir con otro contenedor. ¿Con `push_back` recorriendo desde el final, o con `insert` al frente recorriendo desde el principio?
- Comparar dos `string` con `==` ya está resuelto por la biblioteca.

Ejercicio en parejas: al revés II

Una solución

```
string alReves(string s) {  
    string r;  
    int i = (int) s.size() - 1;  
    while (i >= 0) {  
        r.push_back(s[i]);  
        i = i - 1;  
    }  
    return r;  
}
```

- alReves de reconocer es igual a reconocer: la comparación con == da verdadero. Salida real: al revés: arutcurtse y reconocer al revés es igual.

Ejercicio en parejas: al revés III

- Costo: recorrer desde el final con `push_back` hace n pasos de $O(1)$, $\Theta(n)$ en tiempo, y $\Theta(n)$ de espacio para la cadena nueva. Con `insert` al frente cada inserción corre lo ya escrito y el tiempo sube a $\Theta(n^2)$ con el mismo espacio. vocales: $\Theta(n)$ en tiempo y $\Theta(1)$ de espacio aparte de la cadena.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue**
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación

Los mismos contratos, con los nombres de la biblioteca I

contrato del curso	stack<T>	queue<T>
apilar / encolar	push(x)	push(x)
desapilar / desencolar	pop()	pop()
tope / frente	top()	front()
tamano, vacia	size(), empty()	size(), empty()

- pop no devuelve el valor, igual que desapilar: se lee con top antes.
- top, front y pop sobre un contenedor vacío **no** avisan: no hay at para ellos. La precondition $n > 0$ sigue siendo del llamador, y se vigila con empty() antes de llamar.
- #include <stack> y #include <queue>.

Paréntesis balanceados con stack I

```
bool balanceado(const char *s) {  
    stack<char> p;  
    bool ok = true;  
    int i = 0;  
    while (ok && s[i] != '\0') {  
        if (s[i] == '(' || s[i] == '[') {  
            p.push(s[i]);  
        } else if (p.empty()) {  
            ok = false;  
        } else if (cierra(p.top(), s[i])) {  
            p.pop();  
        } else {  
            ok = false;  
        }  
        i = i + 1;  
    }  
    return ok && p.empty();  
}
```

Paréntesis balanceados con stack II

- Cinco palabras cambiaron respecto a la versión sobre Pila: `stack<char>`, `push`, `empty`, `top`, `pop`. La lógica, las precondiciones y la traza son las mismas.

Recorrer una queue dando la vuelta I

```
// imprime del frente al final y deja la cola como estaba
void imprimir(queue<int> &c) {
    int i = 0;
    int n = (int) c.size();
    while (i < n) {
        int x = c.front();
        printf(" %d", x);
        c.pop();
        c.push(x);
        i = i + 1;
    }
    printf("\n");
}
```

- Con `c = <7 2 9>` imprime 7 2 9; tras un `pop`, imprime 2 9 y `size()` vale 2.

Recorrer una queue dando la vuelta II

- No hay forma de leer el segundo elemento de una queue sin sacar el primero. La biblioteca no agrega operaciones al contrato; solo lo implementa.

Costos de stack y queue

push, pop, top, front, size y empty cuestan $O(1)$; la estructura ocupa $\Theta(n)$. Por eso balanceado sigue en $\Theta(n)$ de tiempo y $O(n)$ de espacio, y este imprimir en $\Theta(n)$ de tiempo y $\Theta(1)$ de espacio aparte de la cola: son los mismos costos que con Pila y Cola, porque el contrato y lo que cobra cada operación son los mismos.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos**
- 7 Lo que cobra cada operación

Un rango y un algoritmo I

```
#include <algorithm>
using std::sort;      using std::reverse;  using std::count;
using std::min_element; using std::max_element;

vector<int> v = {9, 4, 7, 1, 4, 8};
sort(v.begin(), v.end());           // < 1 4 4 7 8 9 >
reverse(v.begin(), v.end());        // < 9 8 7 4 4 1 >
int c = count(v.begin(), v.end(), 4); // 2
int menor = *min_element(v.begin(), v.end()); // 1
int mayor = *max_element(v.begin(), v.end()); // 9
```

La forma de todos

Un algoritmo recibe un **rango**: desde dónde y hasta dónde.
v.begin() señala el primer elemento y v.end() señala *justo después* del último. El rango cubre el vector completo.

Un rango y un algoritmo II

- `min_element` devuelve la **posición** del menor, no el menor; el * adelante lee lo que hay en esa posición. Qué es exactamente esa posición es el tema de la sesión de iteradores.
- Stroustrup, capítulo 32. Todos están en `<algorithm>`.

Buscar: find y binary_search I

```
vector<int> v = {1, 4, 4, 7, 8, 9};           // ordenado

vector<int>::iterator it = find(v.begin(), v.end(), 8);
if (it != v.end()) {
    printf("8 esta en la posicion %d\n", (int) (it -
        ↪ v.begin()));
}
it = find(v.begin(), v.end(), 3);
printf("3 %s\n", it == v.end() ? "no esta" : "esta");

bool hay7 = binary_search(v.begin(), v.end(), 7);    // true
bool hay5 = binary_search(v.begin(), v.end(), 5);    // false
```

- find recorre de principio a fin y devuelve la posición del primer igual, o v.end() si no lo halla. Restar v.begin() convierte la posición en índice: el 8 está en la 4.

Buscar: find y binary_search II

- `binary_search` responde solo sí o no, en muchos menos pasos: parte el rango por la mitad cada vez. A cambio, **exige** que el vector esté ordenado.

Buscar: find y binary_search III

Pregunta

¿Qué responde `binary_search` si el vector no está ordenado?
¿Avisa?

Salida real con {9, 4, 7, 1, 4, 8} sin ordenar

```
sin ordenar, binary_search 7: 0  
sin ordenar, find 7: esta
```

- El 7 está y `binary_search` dice que no. No avisa: la precondition “ordenado” es del llamador, como $q \neq 0$ era del que llamaba a crear. Un algoritmo rápido con la precondition rota es un algoritmo que miente.
- `find` no exige nada y por eso acierta; cobra recorrer todo.

Ordenar con un criterio propio I

Problema

Un vector de estudiantes con nombre y nota. Ordenarlo de mayor a menor nota y, en empate, por nombre alfabético.

```
class Estudiante {  
public:  
    string nombre;  
    double nota;  
    Estudiante(string n, double x) {  
        nombre = n;  
        nota = x;  
    }  
};
```

Ordenar con un criterio propio II

- `sort` sabe ordenar enteros porque sabe comparar enteros. Con estudiantes hay que decirle **quién va antes**: una función que recibe dos y responde verdadero si el primero debe quedar antes que el segundo.

Ordenar con un criterio propio III

Ejercicio en parejas

Escriba `bool` antes(`Estudiante a`, `Estudiante b`) con ese criterio. Antes de escribir, responda: con notas iguales, ¿qué devuelve? Con notas distintas, ¿importa el nombre?

Ordenar con un criterio propio IV

Una solución

```
// dice si a debe ir antes que b
bool antes(Estudiante a, Estudiante b) {
    bool r = false;
    if (a.nota != b.nota) {
        r = a.nota > b.nota;
    } else {
        r = a.nombre < b.nombre;
    }
    return r;
}
```

Ordenar con un criterio propio V

```
vector<Estudiante> g;  
g.push_back(Estudiante("Sara", 3.8));  
g.push_back(Estudiante("Luis", 4.5));  
g.push_back(Estudiante("Ana", 3.8));  
g.push_back(Estudiante("Juan", 4.1));  
sort(g.begin(), g.end(), antes);
```

Salida real

```
Luis  4.5  
Juan  4.1  
Ana   3.8  
Sara  3.8
```

Ordenar con un criterio propio VI

- El tercer argumento de `sort` es la función. Con $>$ en la nota queda descendente; con $<$ en el nombre, el empate 3.8 pone a Ana antes que a Sara.
- Un criterio que responda verdadero para `antes(a, b)` y también para `antes(b, a)` rompe a `sort`: el resultado puede ser cualquiera. Por eso el empate se resuelve con $<$ estricto y no con $\leq$.

Contenido I

- 1 Lo que dejaron los contratos
- 2 Qué es la STL
- 3 vector
- 4 string
- 5 stack y queue
- 6 Los primeros algoritmos
- 7 Lo que cobra cada operación**

Lo que la documentación dice que cuesta I

Operación	Tiempo	Espacio aparte del contenedor
<code>v[p]</code> , <code>at</code> , <code>size</code> , <code>empty</code>	$O(1)$	$O(1)$
<code>push_back</code> , <code>pop_back</code>	$O(1)$ amortizado	$O(1)$
<code>insert</code> , erase en la posición p	$O(n - p)$	$O(1)$
<code>string</code> : <code>+</code>	$O(n)$	$O(n)$, la cadena nueva
<code>string</code> : <code>==</code> , <code><</code>	$O(n)$	$O(1)$
<code>stack</code> y <code>queue</code> : todas	$O(1)$	$O(1)$
<code>find</code> , <code>count</code> , <code>min_element</code> , <code>reverse</code>	$\Theta(n)$	$O(1)$
<code>binary_search</code>	$O(\log n)$	$O(1)$
<code>sort</code>	$O(n \log n)$	$O(\log n)$, la recursión

- Las de $O(1)$ tocan una casilla calculada o un extremo; las de $O(n - p)$ corren lo que sigue; `find` y compañía recorren una vez; `binary_search` parte por la mitad y por eso exige orden; `push_back` es “amortizado” porque de vez en cuando copia todo a un arreglo mayor.

Lo que la documentación dice que cuesta II

Espacio de los contenedores

vector, string, stack y queue ocupan $\Theta(n)$ para n elementos. El vector reserva de más para que push_back no copie cada vez: su capacidad puede superar su tamaño, hasta el doble.

- revertir con insert al frente hace n inserciones de costo hasta n : del orden de n^2 . Con push_back desde el final, n . Mismo contrato, mismo resultado, costo distinto: eso es lo que se escoge al programar.
- Estos costos salen de cómo está guardado el vector, un arreglo que crece. Es justo lo que vamos a construir a mano en las sesiones de implementación, y ahí cada fila de esta tabla se va a demostrar, incluido el “amortizado” de push_back.

Propuestos para la casa I

- 1 Particionar con queue.** Reescriba `particionar` de la sesión de contratos sobre `queue<int>`. Cuente cuántas palabras cambiaron.
- 2 La papa caliente con queue.** Lo mismo con `ultimo(n, k)`. Verifique que con $n = 5$ y $k = 3$ queda 4.
- 3 Mediana.** Dado un `vector<int>`, devuelva la mediana usando `sort`. ¿Qué costo tiene? ¿Hace falta ordenar todo para hallarla?
- 4 Frecuencias.** Dado un vector de palabras (`vector<string>`), imprima cada palabra distinta con el número de veces que aparece. Pista: ordene primero; después el recuento es un solo recorrido.
- 5 Ordenar al revés.** Escriba el criterio para que `sort` deje un `vector<int>` de mayor a menor, y compare con `sort` seguido de `reverse`.

Resumen I

- La STL trae los contratos ya implementados y parametrizados por tipo: `vector<int>`, `stack<char>`, `queue<int>`.
- `vector` es el TAD Lista: `push_back`, `insert`, `erase`, `[]`, `size`. Las funciones escritas contra el contrato se traducen cambiando nombres.
- `at` vigila la precondition y `[]` no. `top`, `front` y `pop` tampoco: $n > 0$ la asegura el llamador con `empty()`.
- Los contenedores se pasan por referencia; por valor se copia todo.
- Los algoritmos reciben un rango `[begin, end)`: `sort` con criterio propio, `find` sin condiciones, `binary_search` solo sobre orden.

Resumen II

Lo que sigue

Los iteradores: qué es de verdad `v.begin()`, cómo recorrer cualquier contenedor con ellos y los algoritmos que faltan; y el repaso para el primer parcial.

Referencias I



B. Stroustrup. *The C++ Programming Language*, 4.^a ed. Addison-Wesley, 2013. Capítulo 31, contenedores de la STL; capítulo 32, algoritmos de la STL.



cppreference. `std::vector`, `std::stack` y `std::sort`.
<https://en.cppreference.com/w/cpp/container/vector>,
<https://en.cppreference.com/w/cpp/container/stack>,
<https://en.cppreference.com/w/cpp/algorithm/sort>.



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 2, tipos abstractos de datos.